



Ayudantía 02: IP-Cores y Operaciones

Ayudante: Ernesto Ferrante - ernesto.ferrante@uc.cl
Profesor: Michel Rozas Fernández - mvrozas@uc.cl

Objetivo de la ayudantía

- Ejercitar el uso de periféricos (*IP-Cores*) dentro de un *Block Design* en Vivado.
- Reforzar los conceptos de RAM, máquina de estados (SM) y ALU.
- Practicar la implementación de operaciones aritméticas y lógicas en VHDL.

Actividades previas a la ayudantía

Todo lo de esta sección debe venir desarrollado antes de la ayudantía. El tiempo de la sesión se destinará por completo al ejercicio propuesto de la última sección, así que se parte del supuesto de que su diseño ya funciona en la tarjeta.

Ambas actividades previas están resueltas paso a paso en el video de la ayudantía 02, grabado el año 2023 por la ex ayudante del curso **Catalina Sierra**, disponible en **este video**. **Apoyarse en el video es opcional:** puede seguirlo completo, usarlo sólo como referencia para destrabar un punto puntual, o resolver las actividades por su cuenta.

Además, los *IP-Cores* utilizados en dicha ayudantía se encuentran disponibles en el repositorio del curso **en este enlace**.

1. Diagrama de bloques: RAM, SM y ALU

Se debe construir el diagrama de bloques compuesto por una memoria RAM, una máquina de estados (SM) y una ALU, tal como se muestra en la Figura 1, generar el *bitstream* y cargarlo en la tarjeta.

2. Nuevas operaciones en la ALU: multiplicación y división

Sobre ese mismo diseño deberán incorporarse dos nuevas operaciones en la ALU: multiplicación y división. Como guía, considere los siguientes pasos:

- Abrir el proyecto donde se desarrolló el código VHDL de la ALU para editarlo de acuerdo con lo solicitado.

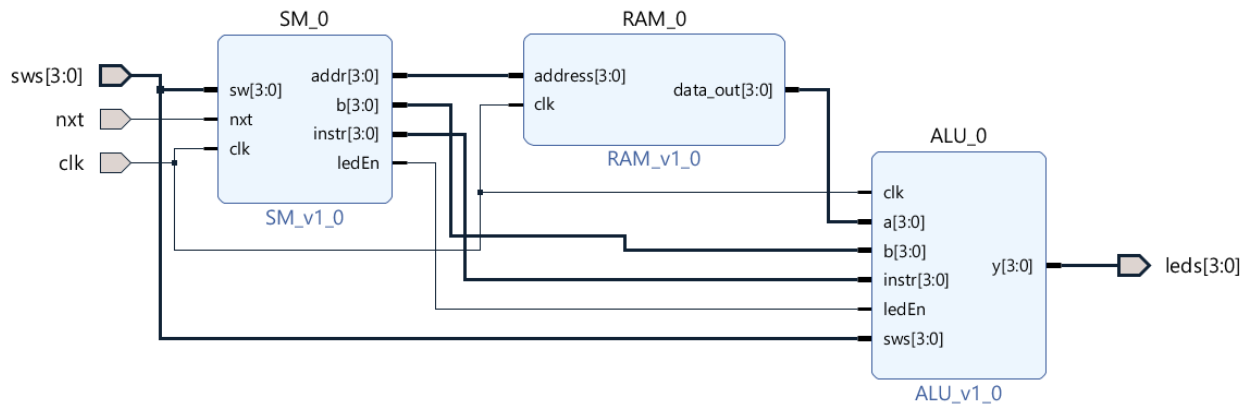


Figura 1: Diagrama de bloques de la ayudantía 02

- Para implementar las operaciones de multiplicación y división, será necesario realizar conversiones de tipos de datos, ya que estas operaciones deben efectuarse sobre valores numéricos enteros. Por esta razón, se debe tener especial cuidado con las conversiones, de modo que el resultado final pueda volver a representarse como un número binario.
- Una vez finalizada la edición, se debe reempaquetar el *IP-Core* de la ALU. Esto se realiza desde el proyecto de la ALU, en la ventana *Package IP*, sección *Review and Package*, seleccionando la opción *Re-Package IP*. Luego, en el *Block Design* del proyecto completo, será necesario actualizar los IP utilizados.

Actividades durante la ayudantía: ejercicio propuesto

El siguiente ejercicio fue propuesto y será desarrollado en la sesión por el ayudante de este semestre, **Ernesto Ferrante**. Es el trabajo que se hará durante la ayudantía, partiendo desde el diseño que usted ya trae funcionando con multiplicación y división.

Consiste en modificar tanto la RAM como la ALU para trabajar con nuevas operaciones arbitrarias.

RAM

En el *IP-Core* de la RAM se deberá modificar la lista de valores previamente almacenados. En lugar de utilizar valores aleatorios, ahora se considerarán valores entre 0 y 15 distribuidos de forma arbitraria, como se muestra en la figura siguiente.

ALU

En el *IP-Core* de la ALU se deberán modificar las operaciones ejecutadas según el código de instrucción, de acuerdo con la Tabla 1.

0	8	10	4
12	2	5	15
14	7	3	13
6	9	11	1

Figura 2: Elementos almacenados en la RAM

Cuadro 1: Tabla de códigos de la ALU

Código	Operación
0000	$y \leq a$
0001	$y \leq b$
0010	$y \leq a + 1$
0011	$y \leq b - 1$
0100	$y \leq a + b$
0101	$y \leq a - b$
0110	$y \leq \frac{a + b}{2}$
0111	$y \leq \frac{a + a + b}{2}$
1000	$y \leq \text{NOT } a$
1001	$y \leq a \text{ AND } b$
1010	$y \leq \frac{a + b + b}{2}$
1011	$y \leq a - b $
1100	$y \leq \frac{a \cdot b}{2}$
1101	$y \leq \frac{a \cdot b + a}{2}$
1110	$y \leq \frac{a \cdot b + b}{2}$
1111	$y \leq \frac{a \cdot b + a + b}{2}$

Entrega y bonificación

El desarrollo del ejercicio propuesto se sube a **Canvas el mismo día de la ayudantía, hasta las 14:50**. Entregarlo dentro de plazo otorga **una décima (+0,1)** en la nota del **Proyecto 1**.

Ayuda

- Recuerde reempaquetar el *IP-Core* luego de realizar la edición. Posteriormente, refresque los *IP-Cores* utilizados en el *Block Design*.
- Para implementar las operaciones propuestas, puede ser útil considerar las siguientes funciones:
 - `std_logic_vector`: se utiliza para representar señales binarias en VHDL. También permite convertir un resultado numérico nuevamente a formato binario para asignarlo a una salida.
 - `to_unsigned`: convierte un número entero a tipo `unsigned`, indicando además la cantidad de bits deseada.
 - `to_integer`: convierte una señal numérica, por ejemplo de tipo `unsigned`, a un valor entero para poder realizar operaciones aritméticas más fácilmente.
 - `unsigned`: interpreta un `std_logic_vector` como un número sin signo, permitiendo realizar operaciones aritméticas sobre él.